

List (part 2)

Syntax: `[item1, item2, item3, item4]`
 ↑
 can be any data type

Indexing: get the item based on index
 syntax: `list[index]` * index starts at 0

Update: Update item based index
 syntax: `list[index] = new_value`

List Operators

- `+` : concatenate two lists together and you get back a new list.

`[1, 2, 3] + [4, 5] → [1, 2, 3, 4, 5]`

- `*` : concatenate the list by itself n times

`[1, 2] * 3 → [1, 2, 1, 2, 1, 2]`

- `in` → check whether the item is in the list? → True False

`(12 in [1, 2, 12, 13]) → True`

~~("AB" in [1, "A", "B", "C"])~~ → False

Ex. check user's id

ID = [123, 456, 789, ~~987~~]

user_id = ⁹⁸⁷int(input("id: ")) e.g. user_id = 987

if (user_id ⁹⁸⁷in ID) → True
print("You're in.") → You're in.

else:
print("Not found")

List Methods

↳ method is a function for data type.

Syntax: list.method_name()

(*)

1).append('item') method

↳ it adds/appends the item to the end of the list.

[1, 2, 3].append(5)

[1, 2, 3, 5]

* it simply adds to the list and not return anything.

* It simply adds to the list and not return anything.
↳ Don't need = .

E.g. `my_list = [1, 2, 3]`
don't need my_list = ... → `my_list.append(5)`
`print(my_list)` → `[1, 2, 3, 5]`
`my_list.append(7)` ✓
`my_list.append(10)` ✓
`print(my_list)` → `[1, 2, 3, 5, 7, 10]`

EX. create a list from user's input.
`n_input = int(input("n: "))`
start empty → `my_list = []` # empty list
loop to append to the list { `for i in range(n_input):`
 `num = int(input())` ← *get from user*
 `my_list.append(num)`
`print(my_list)`

• `insert(index, item)` method
↳ insert the item at the index and then you push everything back one index
`[10, 20, 30, 40].insert(1, 55)`
Diagram: A red bracket under the list [10, 20, 30, 40] indicates that elements from index 1 onwards are shifted one position to the right to make space for the new element at index 1.

[10, 55, 20, 30, 40]

• remove(item) method

↳ remove the first appearance of the item in the list and if the item not in the list, it will be error!

[11, ~~20~~, 30, 20].remove(20)

[11, 30, 20]

[1, 2, 3].remove(5) → Error!

Cuz 5 is not in the list!

Need to check in first

my_list = [1, 2, 3]

item = int(input())

if item in my_list:

my_list.remove(item)

• sort() method

→ sort the list from low to high

[9, 7, 1, 2].sort()

[7, 1, 2].sort()

[1, 2, 5, 7]

Useful built-in functions

- sorted(list, reverse=False)
 - ↳ return a new list sorted from low to high
 - ↳ if reverse=True: high to low

```
.sort()
my_list = [3, 1, 2]
my_list.sort()
print(my_list) → [1, 2, 3]
```

```
sorted(list)
my_list = [3, 1, 2]
my_list = sorted(my_list)
print(my_list) → [1, 2, 3]
```

return a new list

- sum(list) function

↳ return the sum of all items in the list.

↳ e.g. sum([1, 2, 3, 4]) → 10

my_list = [1, 2, 3, 4]

mean = sum(my_list) / len(my_list)

- max(list) function

↳ return the maximum value in the list

e.g. max([1, 5, 7, 3]) → 7

- min(list) function

↳ return the minimum value in the list

↳ return the minimum value in the list
e.g. $\text{min}([1, 5, 7, 3]) \rightarrow 1$

List Referencing

* when you to copy a list, you cannot use $=$.

list1 = [1, 2, 3, 4] →

list2 = list1

list1[0] = 77

print(list1, list2)

[77, 2, 3, 4]

[77, 2, 3, 4]

[~~1~~, 2, 3, 4]

list1

list2

Copy list #1: append()

list1 = [1, 2, 3, 4]

list2 = []

← empty list

for item in list1:

list2.append(item)

← append items from list1

Copy list #2: + operator

list1 = [1, 2, 3, 4]

list2 = [] + list1

* + gives
you a new list

2D list

↳ items in the list can be list.

my_list = [ , ]

print(my_list[0]) → 

my_list = [ [1, 2] ,  [3, 4]]

print(my_list[0]) → [1, 2]

print(my_list[0][0]) → 1

print (my_list[1][1]) → 4

2D list $\rightarrow$ as 2D matrix

my_list = [0, 1, 2, 3],
1 [3, 4, 5],
2 [7, 9, 8]

my_list = [1, 2, 3]

`print(my_list[1][2])` → `5`
 `[3, 4, 5]` ←

`my_list[-1].append(10)` → `[3, 4, 5, 10]`

`print( sum( my_list[1] ) )` → 22
[3, 4, 5, 10]

List Comprehension

1. TFC, a short hand version of

↳ It's a short hand version of
creating a list with append.

append
my_list = [1, 2, 3]
new_list = []
for item in my_list:
 new_list.append(item)

print(new_list) → [1, 2, 3]

List comprehension
my_list = [1, 2, 3]

new_list = [item for item in my_list]

print(new_list) → [1, 2, 3]

Yellow part

can be any expression/code

E.g. Create a new list where all items are squared.
Original = [1, 2, 3, 4]

new_l = []

for num in original:
 new_l.append(num**2)

new_l = [num**2 for num in original]

List comprehension with condition

• Create a new list with only even numbers.

* Create a new list with only even numbers.

nums = [1, 2, 4, 5, 6, 7]

lst = []

for n in nums:

if $n \% 2 == 0$:

lst.append(n)

print(lst) → [2, 4, 6]

lst = [n for n in nums if $n \% 2 == 0$]

List Summary

List = [item1, item2, item3]

Operators: +, *, in

Methods: append, insert, remove, sort
copy list, 2d list, list comprehension

Tuple is an immutable list
↳ same as list but cannot change
more create

↳ Same as list but cannot change
Once Create.

Syntax: tuple = (item1, item2, item3)



* cannot change → Cannot update!

↳ Update [] ✗

↳ append, insert, remove, sort ✗